

Actividad 2

Tamara Hernández Romero

Invalid Date

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import math
```

```
#1 cargar datos
datos = pd.DataFrame({
    'Estudiante': range(1, 11),
    'Horas': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Nota': [3.2, 4.5, 5.0, 6.8, 7.0, 8.5, 9.0, 9.2, 9.8, 10.0]
})
```

REGRESIÓN

```
#generando modelo lineal
x= datos['Horas']
y= datos['Nota']

mediax= np.mean(x)
mediay= np.mean(y)
Sxx= np.sum((x-mediax)**2)
Sxy= np.sum((x-mediax)*(y-mediay))
B1= Sxy/Sxx
B0= mediay- B1*mediax

print('B0: ', B0)
print('B1: ', B1)
y_pred= B1*x + B0
```

```
B0: 3.0533333333333337
B1: 0.7721212121212121
```

```
#visualizacion de regresión
plt.figure(figsize=(10,6))
plt.plot(x,y,"o", alpha=0.6, color="blue", label="notas / horas")
plt.plot(y_pred, linewidth=0.6, color="red",label="Línea de regresión" )
plt.xlabel("horas")
plt.ylabel("notas")
plt.title("Relación entre horas estudiadas y notas de estudiantes")
plt.legend()
plt.show()
```

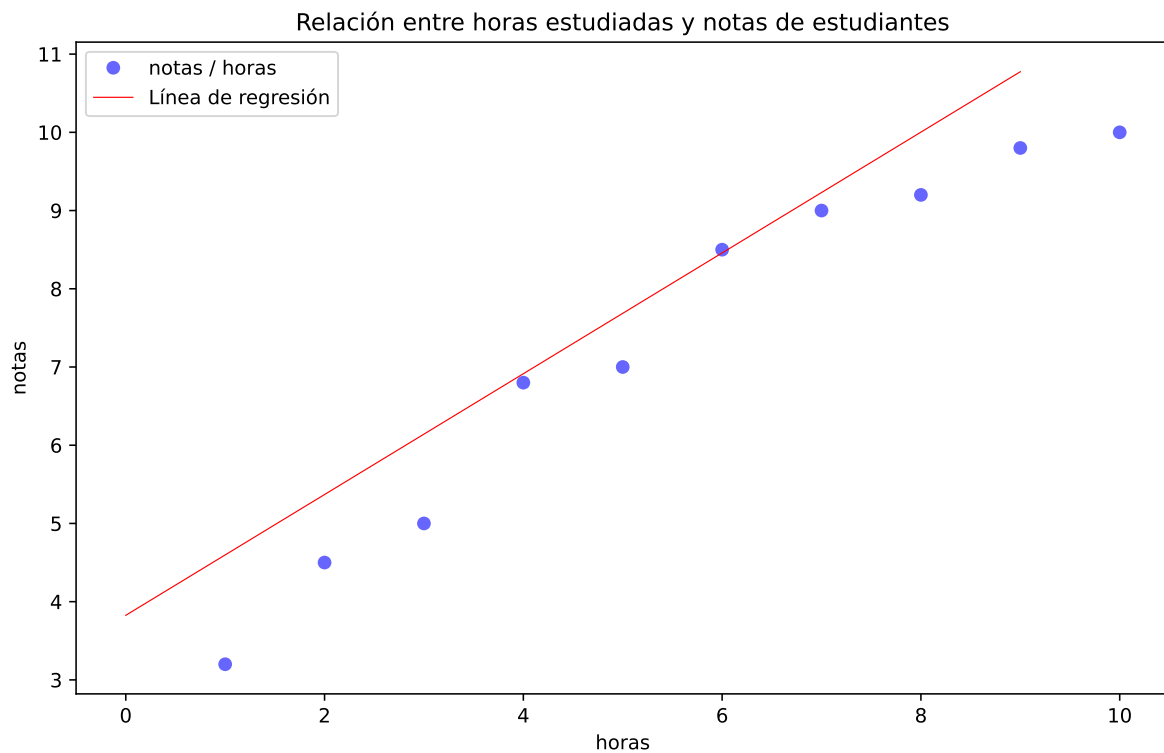


TABLA ANOVA

```
#calculo de componentes columna SS
SSE= np.sum((y-y_pred)**2)
SST= np.sum((y-mediay)**2)
SSR= np.sum((y_pred-mediay)**2)
print('SSE: (variabilidad no explicada)', SSE)
print('SST: (variabilidad total)', SST)
print('SSR: (variabilidad explicada)', SSR)
```

SSE: (variabilidad no explicada) 2.575878787878786
 SST: (variabilidad total) 51.760000000000005
 SSR: (variabilidad explicada) 49.184121212121205

```
#columna df
#donde p son numeros de variables predictoras y n número de obsrvaciones
p=1
dftotal= len(x)-1
dfregresion= p
dfferror= len(x)-p-1
```

```
#columna MS
MSR=SSR/p
MSE=SSE/(len(x)-p-1) #en este caso de regresión lineal simple es tmb la estimación cd la var
print('MSR= ', MSR)
print('MSE= ', MSE)
```

MSR= 49.184121212121205
 MSE= 0.32198484848484826

PRUEBA F y t

```
F= MSR/MSE #estadístico de prueba
print("F= ",F)
```

F= 152.75290574561205

```
anova= {'FUENTE': ['Regresión', 'Error', 'Total'], 'SS': [SSR, SSE, SST], 'df': [dfregresion, dfferror, dftotal]}
pd.DataFrame(anova)
```

	FUENTE	SS	df	MS	F
0	Regresión	49.184121	1	49.184121	152.752906
1	Error	2.575879	8	0.321985	x
2	Total	51.760000	9	X	x

```
tB1= (B1-0)/math.sqrt(MSE/Sxx)
print("Prueba t para B1 t= ", tB1)

#nota para regresión simpplle F=t^2
```

Prueba t para B1 t= 12.35932464763395

INTERVALOS DE CONFIANZA

```
import scipy.stats as st
```

```
alpha=0.05
n=len(x)
df=n-2 #grados de libertad
t_critico = st.t.ppf(1 - alpha / 2, df=df)
```

```
#errores estadar
SE_B1=np.sqrt(MSE/Sxx)
SE_B0=np.sqrt(MSE*((1/n) + (mediax**2/Sxx)))
```

```
ic_B1_inf = B1-t_critico * SE_B1
ic_B1_sup = B1+t_critico * SE_B1

# Intervalo para B0
ic_B0_inf = B0-t_critico * SE_B0
ic_B0_sup = B0+t_critico * SE_B0
```

```
print("intervalo de confianza para B1 superior e inferior: ", ic_B1_sup, ic_B1_inf)
print("intervalo de confianza para B0 superior e inferior: ", ic_B0_sup, ic_B0_inf)
```

```
intervalo de confianza para B1 superior e inferior:  0.9161836717452683 0.6280587524971559
intervalo de confianza para B0 superior e inferior:  3.9472173876199697 2.1594492790466977
```

```
#IC para valor medio
X0=5.5 #valor medio a evaluar

y0_hat=B0+B1*X0
SE_y0 = np.sqrt(MSE*((1/n)+((X0-mediax) **2/Sxx)))

ic_y0_inf = y0_hat-t_critico*SE_y0
ic_y0_sup = y0_hat+t_critico*SE_y0
```

```
print("intervalo de confianza para el valor medio superior e inferior: ", ic_y0_sup, ic_y0_in)
```

```
intervalo de confianza para el valor medio superior e inferior:  7.713787912162373 6.886212087837627
```